

Dynamic Programming: DP on LIS

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Dynamic Programming: DP on LIS	1
1.1	Ending-Here DP: A New Way to Index the Table	1
1.2	Longest Increasing Subsequence	2
1.3	Print the Longest Increasing Subsequence	4
1.4	Longest Increasing Subsequence using Binary Search	6
1.5	Largest Divisible Subset	8
1.6	Longest String Chain	9
1.7	Longest Bitonic Subsequence	11
1.8	Number of Longest Increasing Subsequences	13

Chapter 1

Dynamic Programming: DP on LIS

The longest increasing subsequence asks one question at every position: looking back at everything already placed, what is the best chain I could be extending right now? Once that single question is answered efficiently, an entire family of problems – divisibility chains, string chains, bitonic sequences, counting – falls out as the same question asked with a different notion of “extends.”

1.1 Ending-Here DP: A New Way to Index the Table

Every problem in this chapter uses a state shape not yet seen in this book: $dp[i]$ represents the best answer for a subsequence (or chain) **ending exactly at index i** – not a prefix-based answer like every earlier chapter’s $f(i, \dots)$, which represented the best answer using *only* the first i elements. This “ending-here” indexing is what allows the final answer to be the maximum over *all* $dp[i]$, rather than simply $dp[n]$.

Why the Answer Lives in a Maximum Over the Whole Table, Not One Cell

A longest increasing subsequence can end at *any* position in the array – there is no reason to expect it ends at the last element. So, unlike LCS or Knapsack-family problems where the final answer sits at one specific corner of the table ($dp[n][m]$), here every cell $dp[i]$ is independently a candidate answer, and we must scan the entire filled table for the maximum at the end. This is the single biggest structural difference to watch for in this chapter.

How “Ending-Here” Generalizes Beyond Plain LIS

- **Longest Increasing Subsequence:** $dp[i] = 1 + \max(dp[j])$ over all $j < i$ with $arr[j] < arr[i]$.
- **Largest Divisible Subset (Section 1.5):** identical shape, with the condition $arr[j] < arr[i]$ replaced by $arr[i] \bmod arr[j] = 0$ (after sorting).
- **Longest String Chain (Section 1.6):** identical shape, with the condition replaced by “ $word_j$ becomes $word_i$ after inserting exactly one character” (after sorting by length).
- **Longest Bitonic Subsequence (Section 1.7):** run the ending-here LIS computation **twice** (once forward for increasing runs, once backward for decreasing runs) and combine the two tables at each index.
- **Number of LIS (Section 1.8):** pair $dp[i]$ (the best length ending here) with a second table $count[i]$ (how many subsequences of that best length end here), updated together.

Recognizing this single recurrence shape – extend from any valid earlier “ending-here” cell, under whatever notion of “valid” the problem defines – is the entire content of this chapter.

1.2 Longest Increasing Subsequence

Problem Statement. Given an array arr of n integers, find the length of the **longest strictly increasing subsequence** (elements in order, but not necessarily contiguous).

Pattern Recognition

“Longest increasing subsequence” is the chapter’s foundational problem: at every index, the question is whether it can **extend** some earlier subsequence – which depends on what the *previous chosen element’s value* was, not just which index we’re at.

Deriving the recurrence (pick/not-pick formulation)

Let $f(ind, prevIndex)$ be the length of the longest increasing subsequence achievable from index ind onward, given that the most recently *included* element (if any) is at index $prevIndex$ (or -1 if nothing has been included yet). At each index, either skip it ($f(ind+1, prevIndex)$), or include it – only valid if $prevIndex = -1$ or $arr[prevIndex] < arr[ind]$ – gaining $1 + f(ind+1, ind)$:

$$f(ind, prevIndex) = \max \left(f(ind+1, prevIndex), \text{canTake} \right), \quad \text{canTake} = 1 + f(ind+1, ind) \text{ (if valid)}.$$

Approach 1: Recursion

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lisRecursive(int ind, int prevIndex, vector<int>& arr) {
5     int n = arr.size();
6     if (ind == n) return 0;
7
8     int notTake = lisRecursive(ind + 1, prevIndex, arr);
9     int take = 0;
10    if (prevIndex == -1 || arr[prevIndex] < arr[ind]) {
11        take = 1 + lisRecursive(ind + 1, ind, arr);
12    }
13    return max(notTake, take);
14 }
15
16 int main() {
17     vector<int> arr = {10, 9, 2, 5, 3, 7, 101, 18};
18     cout << lisRecursive(0, -1, arr) << endl; // 4
19     return 0;
20 }
```

Approach 1 Complexity

Time. $O(2^n)$. **Space.** $O(n)$ recursion stack.

Approach 2: Memoization

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lisMemo(int ind, int prevIndex, vector<int>& arr, vector<vector<int>
    >>& memo) {
```

```

5   int n = arr.size();
6   if (ind == n) return 0;
7   int prevShift = prevIndex + 1; // shift by 1 so -1 maps to 0
8   if (memo[ind][prevShift] != -1) return memo[ind][prevShift];
9
10  int notTake = lisMemo(ind + 1, prevIndex, arr, memo);
11  int take = 0;
12  if (prevIndex == -1 || arr[prevIndex] < arr[ind]) {
13      take = 1 + lisMemo(ind + 1, ind, arr, memo);
14  }
15  return memo[ind][prevShift] = max(notTake, take);
16 }
17
18 int main() {
19     vector<int> arr = {10, 9, 2, 5, 3, 7, 101, 18};
20     int n = arr.size();
21     vector<vector<int>> memo(n, vector<int>(n + 1, -1));
22     cout << lisMemo(0, -1, arr, memo) << endl; // 4
23     return 0;
24 }

```

Approach 2 Complexity

Time. $O(n^2)$. **Space.** $O(n^2)$ memo + $O(n)$ recursion stack.

Approach 3: Tabulation (Ending-Here Reformulation)

Why Tabulation Uses a Cleaner, Different-Looking Table

Directly tabulating the $(ind, prevIndex)$ recursion is awkward (the shifted-index bookkeeping is unpleasant to iterate over bottom-up). The standard simplification: define $dp[i]$ = length of the LIS *ending exactly at* index i . This is mathematically equivalent information, just reorganized; it gives a much cleaner loop and is how this problem is conventionally tabulated.

$$dp[i] = 1 + \max_{j < i, \text{arr}[j] < \text{arr}[i]} dp[j] \quad (\text{or } 1 \text{ if no such } j \text{ exists}).$$

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int lisTabulation(vector<int>& arr) {
5      int n = arr.size();
6      vector<int> dp(n, 1); // every element alone is an LIS of length 1
7
8      int best = 1;
9      for (int i = 0; i < n; i++) {
10         for (int j = 0; j < i; j++) {
11             if (arr[j] < arr[i]) {
12                 dp[i] = max(dp[i], 1 + dp[j]);
13             }
14         }
15         best = max(best, dp[i]);
16     }
17     return best;
18 }
19

```

```

20 int main() {
21     vector<int> arr = {10, 9, 2, 5, 3, 7, 101, 18};
22     cout << lisTabulation(arr) << endl; // 4
23     return 0;
24 }

```

Dry run of the tabulation table

Dry Run. Take $arr = [10, 9, 2, 5, 3, 7, 101, 18]$.

i	0	1	2	3	4	5	6	7
$arr[i]$	10	9	2	5	3	7	101	18
$dp[i]$	1	1	1	2	2	3	4	4

$dp[5] = 3$ comes from extending the chain ending at index 3 (value 5, $dp = 2$): 2, 5, 7. $dp[6] = 4$ extends that same chain with 101: 2, 5, 7, 101. The maximum over the whole table is 4.

Approach 3 Complexity

Time. $O(n^2)$ – a nested loop comparing every pair. **Space.** $O(n)$ for the dp array – already minimal; this problem’s natural tabulated form has no further space reduction the way 2D-table problems in earlier chapters did.

A Faster $O(n \log n)$ Algorithm Exists

Section 1.4 develops a completely different $O(n \log n)$ technique for this same problem, using binary search instead of dynamic programming’s pairwise comparison – worth knowing as the production-quality solution once the $O(n^2)$ DP version here is understood.

Key Takeaway

“Ending-here” tables answer their question with a **maximum over the entire filled table**, not a single corner cell – the first new structural lesson of this chapter, and the one to watch for in every section that follows.

1.3 Print the Longest Increasing Subsequence

Problem Statement. Given arr , construct (not just measure the length of) one actual longest increasing subsequence.

Pattern Recognition

Exactly as in the DP on Strings chapter’s Print LCS section, reconstructing the actual solution needs the full tabulation table kept around, plus a **parent pointer** recorded at each index, tracking *which earlier index* extended into it – since the table itself only stores chain *lengths*, not which specific chain produced them.

Understanding the reconstruction

Alongside $dp[i]$ (the LIS length ending at i), maintain $parent[i]$: the index $j < i$ that $dp[i]$ was extended from (or i itself, if $dp[i] = 1$ and nothing extends it). After filling the table, find the index with the maximum dp value, then follow $parent$ pointers backward until reaching an index whose parent is itself, collecting elements along the way.

Algorithm

1. Initialize $dp[i] \leftarrow 1$, $parent[i] \leftarrow i$ for all i .
2. For i from 0 to $n - 1$: for j from 0 to $i - 1$: if $arr[j] < arr[i]$ and $1 + dp[j] > dp[i]$: update $dp[i] \leftarrow 1 + dp[j]$; $parent[i] \leftarrow j$.
3. Find $lastIndex$ maximizing $dp[i]$.
4. Walk backward from $lastIndex$ via $parent$ until $parent[i] = i$, collecting each $arr[i]$ visited; reverse the collected list.

Dry run

Dry Run. Using Section 1.2's table for $arr = [10, 9, 2, 5, 3, 7, 101, 18]$.

$dp = [1, 1, 1, 2, 2, 3, 4, 4]$, with $parent[3] = 2$ (chain 5 extends 2), $parent[5] = 3$ (chain 7 extends 5), $parent[6] = 5$ (chain 101 extends 7). The maximum dp value is 4, first achieved at index 6. Walking backward: $6 \rightarrow 5 \rightarrow 3 \rightarrow 2$, collecting 101, 7, 5, 2; reversing gives [2, 5, 7, 101].

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> printLIS(vector<int>& arr) {
5     int n = arr.size();
6     vector<int> dp(n, 1), parent(n);
7     for (int i = 0; i < n; i++) parent[i] = i;
8
9     int lastIndex = 0;
10    for (int i = 0; i < n; i++) {
11        for (int j = 0; j < i; j++) {
12            if (arr[j] < arr[i] && 1 + dp[j] > dp[i]) {
13                dp[i] = 1 + dp[j];
14                parent[i] = j;
15            }
16        }
17        if (dp[i] > dp[lastIndex]) lastIndex = i;
18    }
19
20    vector<int> result;
21    int cur = lastIndex;
22    while (parent[cur] != cur) {
23        result.push_back(arr[cur]);
24        cur = parent[cur];
25    }
26    result.push_back(arr[cur]);
27    reverse(result.begin(), result.end());
28    return result;
29 }
30
31 int main() {
32     vector<int> arr = {10, 9, 2, 5, 3, 7, 101, 18};
33     for (int x : printLIS(arr)) cout << x << " ";
34     cout << endl; // 2 5 7 101
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. $O(n^2)$ to build the table, plus $O(n)$ for the backward walk. **Space Complexity.** $O(n)$ for the *dp* and *parent* arrays.

Key Takeaway

“Print the actual chain” problems on an ending-here table always need a parent-pointer array recorded *during* the forward fill, then a backward walk from whichever index holds the global maximum – the same reconstruction idea as Print LCS, adapted to this chapter’s different table shape.

1.4 Longest Increasing Subsequence using Binary Search

Problem Statement. Solve Section 1.2 (find the length of the LIS) in $O(n \log n)$ instead of $O(n^2)$.

Pattern Recognition

“Find the length of the LIS, faster than $O(n^2)$ ” is solved not by a better DP table, but by an entirely different technique: maintain a single array *tails*, where *tails*[*k*] is the **smallest possible value** that can end an increasing subsequence of length $k + 1$, discovered so far – and update it for each new element using **binary search** (the Binary Search chapter’s lower bound) instead of scanning all previous elements.

Understanding why *tails* is always sorted

The key invariant: *tails* is always kept sorted in increasing order. This is not obvious at first, but follows by induction: a longer increasing subsequence’s smallest possible tail value can never be smaller than a shorter one’s smallest possible tail value (if it were, that shorter prefix of the longer subsequence would itself be a counterexample to the shorter subsequence’s tail already being minimal). Because *tails* stays sorted, we can binary search it.

Update Rule for Each New Element

For each $x = \text{arr}[i]$, in order: binary search *tails* for the leftmost position *pos* where $\text{tails}[\text{pos}] \geq x$ (this is exactly the Binary Search chapter’s lower bound technique). If no such position exists (x is larger than every current tail), append x (a new, longer subsequence is now possible). Otherwise, replace $\text{tails}[\text{pos}] \leftarrow x$ (x is a *smaller* valid tail for a subsequence of that same length, strictly improving future extension possibilities, without changing the *count* of distinct lengths discovered).

Why replacing (not just appending) preserves correctness

tails’s length, at any point, equals the LIS length found so far among the elements processed – but the actual *values* stored are not necessarily a real subsequence from the array; they are simply the best (smallest) tail achievable for each length. Replacing *tails*[*pos*] with a smaller value x never decreases *tails*’s length (we are not removing entries, only improving one in place), and it can only ever *help* future elements extend a chain of that length, since a smaller tail is easier for a later, possibly-not-much-larger element to exceed. The final length of *tails* is therefore exactly the LIS length.

Algorithm

1. Initialize an empty array *tails*.
2. For each x in *arr*: binary search for the lower bound of x in *tails*. If found at the end (no entry $\geq x$): append x . Else: overwrite that entry with x .
3. Return $|tails|$.

Dry run

Dry Run. Take $arr = [10, 9, 2, 5, 3, 7, 101, 18]$.

- $x = 10$: *tails* empty, append. $tails = [10]$.
- $x = 9$: lower bound of 9 in $[10]$ is index 0. Overwrite. $tails = [9]$.
- $x = 2$: lower bound of 2 in $[9]$ is index 0. Overwrite. $tails = [2]$.
- $x = 5$: lower bound of 5 in $[2]$: none ≥ 5 , append. $tails = [2, 5]$.
- $x = 3$: lower bound of 3 in $[2, 5]$ is index 1 (value 5). Overwrite. $tails = [2, 3]$.
- $x = 7$: lower bound of 7 in $[2, 3]$: none ≥ 7 , append. $tails = [2, 3, 7]$.
- $x = 101$: append. $tails = [2, 3, 7, 101]$.
- $x = 18$: lower bound of 18 in $[2, 3, 7, 101]$ is index 3 (value 101). Overwrite. $tails = [2, 3, 7, 18]$.

Final length: 4 – matching Section 1.2’s answer (note $tails = [2, 3, 7, 18]$ is not necessarily a real subsequence of *arr*, but its *length* is correctly the LIS length).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int lisBinarySearch(vector<int>& arr) {
5     vector<int> tails;
6
7     for (int x : arr) {
8         int pos = lower_bound(tails.begin(), tails.end(), x) - tails.
9             begin();
10        if (pos == (int)tails.size()) {
11            tails.push_back(x);
12        } else {
13            tails[pos] = x;
14        }
15    }
16    return tails.size();
17 }
18
19 int main() {
20     vector<int> arr = {10, 9, 2, 5, 3, 7, 101, 18};
21     cout << lisBinarySearch(arr) << endl; // 4
22     return 0;
23 }
```

Complexity Analysis

Time Complexity. Each of the n elements performs one binary search over *tails* (size at most n), giving

$$O(n \log n).$$

Space Complexity. $O(n)$ for the *tails* array.

Key Takeaway

Not every problem's fastest solution is a bigger or cleverer DP table – sometimes, as here, an entirely different paradigm (greedy maintenance of a sorted auxiliary array, queried by binary search) beats DP's polynomial bound outright. Keep this $O(n \log n)$ technique available specifically for when a problem only needs the LIS *length*; it does not directly reconstruct the actual subsequence the way Section 1.3's parent-pointer approach does.

1.5 Largest Divisible Subset

Problem Statement. Given an array of **distinct** positive integers, find the largest subset such that for every pair of elements (a, b) in it, either $a \bmod b = 0$ or $b \bmod a = 0$.

Pattern Recognition

“Largest subset where every pair divides” is Section 1.1's ending-here template with the condition $arr[j] < arr[i]$ replaced by $arr[i] \bmod arr[j] = 0$ – after first **sorting** the array, exactly the same preparatory step LIS uses implicitly (LIS's input is typically already read left to right; here we must sort first to make the pairwise-divisibility condition reduce to a simple left-to-right chain check).

Why sorting first is essential

After sorting, if $arr[k]$ divides $arr[j]$ and $arr[j]$ divides $arr[i]$ (with $k < j < i$ in the sorted order), then $arr[k]$ *automatically* divides $arr[i]$ too (divisibility is transitive). So checking only **consecutive** chain links ($arr[j] \mid arr[i]$ for the immediately preceding chosen element) is sufficient to guarantee *every pair* in the resulting chain satisfies the divisibility condition – exactly mirroring why LIS only checks $arr[j] < arr[i]$ pairwise rather than every triple.

The recurrence

$$dp[i] = 1 + \max_{j < i, arr[i] \bmod arr[j] = 0} dp[j] \quad (\text{or } 1 \text{ if no such } j).$$

Algorithm (with reconstruction)

Exactly Section 1.3's parent-pointer approach, with the extension condition swapped to divisibility.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> largestDivisibleSubset(vector<int> arr) {
5     int n = arr.size();
6     sort(arr.begin(), arr.end());
7
8     vector<int> dp(n, 1), parent(n);

```

```

9     for (int i = 0; i < n; i++) parent[i] = i;
10
11     int lastIndex = 0;
12     for (int i = 0; i < n; i++) {
13         for (int j = 0; j < i; j++) {
14             if (arr[i] % arr[j] == 0 && 1 + dp[j] > dp[i]) {
15                 dp[i] = 1 + dp[j];
16                 parent[i] = j;
17             }
18         }
19         if (dp[i] > dp[lastIndex]) lastIndex = i;
20     }
21
22     vector<int> result;
23     int cur = lastIndex;
24     while (parent[cur] != cur) {
25         result.push_back(arr[cur]);
26         cur = parent[cur];
27     }
28     result.push_back(arr[cur]);
29     reverse(result.begin(), result.end());
30     return result;
31 }
32
33 int main() {
34     vector<int> arr = {1, 16, 7, 8, 4};
35     for (int x : largestDivisibleSubset(arr)) cout << x << " ";
36     cout << endl; // 1 4 8 16
37     return 0;
38 }

```

Dry run

Dry Run. Take $arr = [1, 16, 7, 8, 4]$, sorted: $[1, 4, 7, 8, 16]$.

$dp = [1, 2, 1, 3, 4]$: $4 \bmod 1 = 0 \Rightarrow dp[1] = 2$; $8 \bmod 4 = 0 \Rightarrow dp[3] = 3$ (extending the chain $1, 4, 8$); $16 \bmod 8 = 0 \Rightarrow dp[4] = 4$ (extending $1, 4, 8, 16$). Note 7 extends nothing (no earlier sorted element divides it), so $dp[2] = 1$. The maximum is 4, giving the subset $[1, 4, 8, 16]$.

Complexity Analysis

Time Complexity. $O(n \log n)$ to sort, plus $O(n^2)$ to fill the table: $O(n^2)$ overall. **Space Complexity.** $O(n)$ for dp and $parent$.

Key Takeaway

Whenever a pairwise condition is **transitive** (if A relates to B and B relates to C then A relates to C), sorting first and then checking only consecutive chain links is enough – the exact property that lets both LIS’s “less than” and this problem’s “divides” collapse an all-pairs condition down to a simple left-to-right ending-here DP.

1.6 Longest String Chain

Problem Statement. Given a list of words, a **string chain** is a sequence $word_1, word_2, \dots, word_k$ where each $word_i$ (for $i > 1$) can be formed by inserting exactly one character, anywhere, into $word_{i-1}$. Find the length of the longest possible chain using the given words.

Pattern Recognition

A third instance of Section 1.1's ending-here template: sort by **length** (the natural analogue of LIS's numeric ordering), and replace the extension condition with “removing exactly one character from $word_i$ yields exactly $word_j$.”

The predecessor check

A word w_j is a valid predecessor of w_i exactly when $|w_i| = |w_j| + 1$ and deleting some single character from w_i produces w_j exactly. Checking this directly: try deleting each character of w_i in turn (one position at a time) and see if any resulting string equals w_j .

The recurrence

After sorting words by increasing length:

$$dp[i] = 1 + \max_{j < i, \text{isPredecessor}(w_j, w_i)} dp[j] \quad (\text{or } 1 \text{ if no such } j).$$

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isPredecessor(string& shorter, string& longer) {
5     if (longer.size() != shorter.size() + 1) return false;
6     int i = 0, j = 0;
7     bool skippedOne = false;
8     while (i < (int)longer.size() && j < (int)shorter.size()) {
9         if (longer[i] == shorter[j]) {
10             i++; j++;
11         } else if (!skippedOne) {
12             i++; skippedOne = true;
13         } else {
14             return false;
15         }
16     }
17     return true;
18 }
19
20 int longestStringChain(vector<string> words) {
21     sort(words.begin(), words.end(),
22         [](const string& a, const string& b) { return a.size() < b.size(); });
23
24     int n = words.size();
25     vector<int> dp(n, 1);
26     int best = 1;
27
28     for (int i = 0; i < n; i++) {
29         for (int j = 0; j < i; j++) {
30             if (isPredecessor(words[j], words[i]) && 1 + dp[j] > dp[i]) {
31                 dp[i] = 1 + dp[j];
32             }
33         }
34     }
35 }
```

```

34     best = max(best, dp[i]);
35 }
36 return best;
37 }
38
39 int main() {
40     vector<string> words = {"a", "b", "ba", "bca", "bda", "bdca"};
41     cout << longestStringChain(words) << endl; // 4
42     return 0;
43 }

```

Dry run

Dry Run. Take $words = ["a", "b", "ba", "bca", "bda", "bdca"]$, already sorted by length.

$dp["a"] = dp["b"] = 1$. $dp["ba"] = 2$ (predecessor "a" or "b", either gives $1 + 1 = 2$).
 $dp["bca"] = 3$ (predecessor "ba", delete 'c'). $dp["bda"] = 3$ (predecessor "ba", delete 'd').
 $dp["bdca"] = 4$ (predecessor "bca", delete 'd' – or equally "bda", delete 'c'; either extends a length-3 chain to 4). The maximum is 4, realized by the chain "a" $\rightarrow$ "ba" $\rightarrow$ "bca" $\rightarrow$ "bdca".

Complexity Analysis

Time Complexity. $O(n \log n)$ to sort, plus $O(n^2 \cdot L)$ to fill the table ($L = \max$ word length, the cost of each *isPredecessor* check): $O(n^2 \cdot L)$ overall. **Space Complexity.** $O(n)$ for dp .

Key Takeaway

The “sort, then ending-here chain with a custom extension check” template now has three independent instances (numeric order for LIS, divisibility for the previous section, and length-plus-single-insertion here) – a strong signal that this template, not any one specific condition, is the reusable idea worth carrying forward.

1.7 Longest Bitonic Subsequence

Problem Statement. Given an array arr , find the length of the longest **bitonic** subsequence: one that first strictly increases, then strictly decreases (either the increasing or decreasing part may be empty, but not both).

Pattern Recognition

“Increases then decreases” is solved by running Section 1.2’s ending-here LIS computation **twice** – once forward (for the increasing run up to each index) and once backward (for the decreasing run from each index onward) – then combining the two values at every index, since a bitonic sequence’s peak can be at any position.

Deriving the combination formula

Let $inc[i]$ = length of the longest increasing subsequence *ending* at index i (exactly Section 1.2’s table). Let $dec[i]$ = length of the longest decreasing subsequence *starting* at index i – computed identically, but scanning *backward* from the end and requiring $arr[i] > arr[j]$ for $j > i$ (equivalently, LIS run on the array’s mirror image). If index i is the peak of a bitonic sequence, the increasing run contributes $inc[i]$ elements and the decreasing run contributes $dec[i]$ elements,

but $arr[i]$ itself is counted in both, so:

$$bitonic[i] = inc[i] + dec[i] - 1.$$

The answer is $\max_i bitonic[i]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int longestBitonicSubsequence(vector<int>& arr) {
5     int n = arr.size();
6     vector<int> inc(n, 1), dec(n, 1);
7
8     for (int i = 0; i < n; i++) {
9         for (int j = 0; j < i; j++) {
10             if (arr[j] < arr[i]) {
11                 inc[i] = max(inc[i], 1 + inc[j]);
12             }
13         }
14     }
15
16     for (int i = n - 1; i >= 0; i--) {
17         for (int j = n - 1; j > i; j--) {
18             if (arr[j] < arr[i]) {
19                 dec[i] = max(dec[i], 1 + dec[j]);
20             }
21         }
22     }
23
24     int best = 0;
25     for (int i = 0; i < n; i++) {
26         best = max(best, inc[i] + dec[i] - 1);
27     }
28     return best;
29 }
30
31 int main() {
32     vector<int> arr = {1, 11, 2, 10, 4, 5, 2, 1};
33     cout << longestBitonicSubsequence(arr) << endl; // 6
34     return 0;
35 }
```

Dry run

Dry Run. Take $arr = [1, 11, 2, 10, 4, 5, 2, 1]$.

$inc = [1, 2, 2, 3, 3, 4, 2, 1]$ (the run 1, 2, 4, 5 ending at index 5 gives $inc[5] = 4$). $dec = [1, 5, 2, 4, 3, 3, 2, 1]$ (the run 11, 10, 5, 2, 1 starting at index 1 gives $dec[1] = 5$). $bitonic[1] = inc[1] + dec[1] - 1 = 2 + 5 - 1 = 6$, the maximum – realized by the sequence 1, 11, 10, 5, 2, 1, increasing from 1 to the peak 11 (index 1) and then decreasing.

Complexity Analysis

Time Complexity. $O(n^2)$ – two nested-loop passes. **Space Complexity.** $O(n)$ for inc and dec .

Key Takeaway

When a problem's structure splits naturally into a “before the turning point” half and an “after the turning point” half, compute each half's ending-here table independently (one forward, one backward) and combine them at every candidate turning point – a pattern that generalizes well beyond bitonic sequences to any problem with a single peak or pivot.

1.8 Number of Longest Increasing Subsequences

Problem Statement. Given *arr*, find the **number of distinct** longest increasing subsequences (not just their shared maximum length).

Pattern Recognition

This closing problem of the chapter pairs Section 1.2's ending-here *dp* table with a second, parallel table *count[i]*: **how many** subsequences of the maximum length achieve *dp[i]* – updated using the same “exists / count” duality first introduced in the DP on Subsequences chapter (there distinguishing whether a target sum is *achievable* versus *how many ways* achieve it).

Deriving the count update rule

While filling *dp[i]* in the usual way, also track *count[i]*, initialized to 1 (every index is trivially one subsequence of length 1, by itself). For each $j < i$ with $arr[j] < arr[i]$:

- If $dp[j] + 1 > dp[i]$: a **strictly longer** chain through j has just been discovered. Update $dp[i] \leftarrow dp[j] + 1$ and **reset** $count[i] \leftarrow count[j]$ (every previous, shorter-chain count at i is now obsolete – it counted chains of a length that's no longer the best).
- If $dp[j] + 1 = dp[i]$: this is **another equally good** way to reach the current best length at i . **Accumulate**: $count[i] += count[j]$.

After filling both tables, let $maxLen = \max_i dp[i]$. The answer is $\sum_i: dp[i]=maxLen count[i]$ (every index whose best chain achieves the global maximum length contributes its count).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int numberOfLIS(vector<int>& arr) {
5     int n = arr.size();
6     vector<int> dp(n, 1), count(n, 1);
7
8     for (int i = 0; i < n; i++) {
9         for (int j = 0; j < i; j++) {
10             if (arr[j] < arr[i]) {
11                 if (dp[j] + 1 > dp[i]) {
12                     dp[i] = dp[j] + 1;
13                     count[i] = count[j];
14                 } else if (dp[j] + 1 == dp[i]) {
15                     count[i] += count[j];
16                 }
17             }
18         }
19     }
20 }
```

```

18     }
19 }
20
21 int maxLen = *max_element(dp.begin(), dp.end());
22 int total = 0;
23 for (int i = 0; i < n; i++) {
24     if (dp[i] == maxLen) total += count[i];
25 }
26 return total;
27 }
28
29 int main() {
30     vector<int> arr = {1, 3, 5, 4, 7};
31     cout << numberOfLIS(arr) << endl; // 2
32     return 0;
33 }

```

Dry run

Dry Run. Take $arr = [1, 3, 5, 4, 7]$.

i	0	1	2	3	4
$arr[i]$	1	3	5	4	7
$dp[i]$	1	2	3	3	4
$count[i]$	1	1	1	1	2

At $i = 3$ (value 4): only $j = 0, 1$ qualify ($1 < 4, 3 < 4$); the best is through $j = 1$ ($dp[1] + 1 = 3$), giving $dp[3] = 3$, $count[3] = count[1] = 1$. At $i = 4$ (value 7): $j = 2$ gives $dp[2] + 1 = 4$ (new max, reset $count[4] = count[2] = 1$); then $j = 3$ also gives $dp[3] + 1 = 4$ (ties the current best, accumulate: $count[4] += count[3] = 1 + 1 = 2$). $maxLen = 4$, achieved only at $i = 4$, contributing $count[4] = 2$: the two LIS of length 4 are 1, 3, 5, 7 and 1, 3, 4, 7.

Complexity Analysis

Time Complexity. $O(n^2)$. **Space Complexity.** $O(n)$ for dp and $count$.

Key Takeaway

This chapter closes exactly where the DP on Subsequences chapter's “exists / count / optimize” framing predicted it would: once a length-optimizing ending-here table exists, a parallel counting table updated by the same reset-on-strict-improvement, accumulate-on-tie rule answers “how many” for free. The same two-table pairing pattern is worth trying on any optimization DP whenever a follow-up problem asks for a count instead of (or in addition to) the optimum itself.